Team 2 S&P 500 notebook: with edits

File: sp500_panel-2.ipynb (sp500_v2 repo, commit 888b0c5). Original notebook and edits by Team 2, 14 Oct 2026. Edit markers are shown here as TEAM 2 EDIT / ORIGINAL.

White = original code (unchanged)

Green = new or changed code

Orange = the original line, kept as a comment so the change is visible

Dark green bar = start / end marker of each edit

Blue = explanation text (markdown cells)

Note: the CSV is **not** switched off. The notebook still writes sp500_panel.csv, and the Parquet files are written **in addition**, so nothing that reads his CSV breaks.

The 6 edits at a glance

#	What changed	Why
1	SEC email read from .env	Personal email no longer in the code
2	Yahoo batch 100 → 25	Fewer rate-limit drops
3	Retry missing / behind tickers (3 tries)	7 tickers were silently missing on 14 Oct
4	Parquet file per year in data/<date>/	Typed, compressed, loads to BigQuery partitioned; scales better than one CSV
5	Plain screen table	Styled version needed an extra package (jinja2)
6	Completeness check with exit code	Dagster marks the run FAILED if >5 tickers missing

Full notebook

S&P 500 daily panel: 5 years of prices, sector and per-share fundamentals in one CSV

Run the cells top to bottom (or "Run All"). The result is **one file, `sp500_panel.csv`**, with one row per ticker per trading day:

- daily open, high, low, close, adjusted close and volume (5 years)
- GICS sector and sub-industry, and the date each stock joined the index
- the per-share fundamentals that were public on that day (EPS, cash flow per share, balance sheet per share)
- daily valuation ratios built from the two (market cap, P/E, P/CF, P/B, dividend yield)

Sources: Wikipedia (constituents and sector), SEC EDGAR company-facts API (fundamentals), Yahoo Finance via `yfinance` (prices).

Expect roughly 10 to 15 minutes in total; the SEC step is the slow one (one request per company).

> **Edited by Team 2, 14 Oct 2026.** Original notebook by **Team 2**. Every change is marked `# >>> TEAM 2 EDIT` … `# <<< END TEAM 2 EDIT`, with the original line kept as a `# ORIGINAL:` comment. Changes: SEC email from `.env`; Yahoo batch 100 → 25; retry tickers missing the latest day; Parquet per year; plain screen table; final completeness check.

Code cell 1 Original, unchanged

```
# Run once, then you can comment it out again
# %pip install pandas requests lxml yfinance
```

0. Settings
Put your own name and email in `SEC_USER_AGENT`. The SEC blocks requests that do not identify themselves.

Code cell 2 EDITED

```
import time
from io import StringIO

import numpy as np
import pandas as pd
import requests
import yfinance as yf

# >>> TEAM 2 EDIT (14 Oct 2026): read the SEC email from .env instead of hard-coding it
# ORIGINAL: SEC_USER_AGENT = "[email removed]"
import os
from dotenv import load_dotenv
load_dotenv()                                          # reads .env in this folder
SEC_USER_AGENT = os.environ["SEC_USER_AGENT"]    # e.g. "Your Name you@example.com"
# <<< END TEAM 2 EDIT
OUT_FILE = "sp500_panel.csv"
COMPRESS = False       # True also writes sp500_panel.csv.gz, a much smaller copy that pandas reads directly
PRICE_YEARS = 5        # price history written to CSV
LOOKBACK_YEARS = 8     # history pulled internally (growth rates need 2 extra years; splits need to be seen)
WIKI_URLS = ["https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"]
MIN_ROWS = 400         # the components table has about 503 rows; anything much shorter is a different table

# Tickers whose filings report per-share figures for a different share class.
# Berkshire reports per Class A share; one A share = 1,500 B shares.
SHARE_CLASS_RATIO = {"BRK.B": 1500}
```

```
Q_DAYS = (70, 125)     # length of one fiscal quarter in days (covers 12-, 13-,
14- and 16-week quarters)
FY_DAYS = (350, 380)   # length of one fiscal year in days

# XBRL tags, in order of preference. Companies do not all use the same tag for
the same concept.
TAGS = {
    "eps_diluted": ["EarningsPerShareDiluted", "EarningsPerShareBasicAndDilute
d"],
    "eps_basic":   ["EarningsPerShareBasic", "EarningsPerShareBasicAndDilute
d"],
    "revenue":     ["RevenueFromContractWithCustomerExcludingAssessedTax", "Rev
enues", "SalesRevenueNet",
                    "RevenueFromContractWithCustomerIncludingAssessedTax", "Rev
enuesNetOfInterestExpense"],
    "net_income":  ["NetIncomeLoss", "ProfitLoss"],
    "cfo":         ["NetCashProvidedByUsedInOperatingActivities",
                    "NetCashProvidedByUsedInOperatingActivitiesContinuingOperat
ions"],
    "capex":       ["PaymentsToAcquirePropertyPlantAndEquipment", "PaymentsToAc
quireProductiveAssets"],
    "dps":         ["CommonStockDividendsPerShareDeclared", "CommonStockDividen
dsPerShareCashPaid"],
    "shares_dil":  ["WeightedAverageNumberOfDilutedSharesOutstanding",
                    "WeightedAverageNumberOfShareOutstandingBasicAndDiluted"],
    "shares_bas":  ["WeightedAverageNumberOfSharesOutstandingBasic",
                    "WeightedAverageNumberOfShareOutstandingBasicAndDiluted"],
    # balance sheet (point-in-time)
    "shares_eop":  ["CommonStockSharesOutstanding"],
    "shares_cover": ["EntityCommonStockSharesOutstanding"],      # 10-Q / 10-K
cover page (dei taxonomy)
    "equity":      ["StockholdersEquity", "StockholdersEquityIncludingPortionAt
tributableToNoncontrollingInterest"],
    "assets":      ["Assets"],
    "liabilities": ["Liabilities"],
    "cash":        ["CashAndCashEquivalentsAtCarryingValue", "CashAndDueFromBan
ks",
                    "CashCashEquivalentsRestrictedCashAndRestrictedCashEquivale
nts"],
    "lt_debt":     ["LongTermDebt", "LongTermDebtNoncurrent", "LongTermDebtAndC
apitalLeaseObligations",
                    "ConvertibleDebtNoncurrent", "ConvertibleNotesPayableNoncur
rent", "SeniorNotes"],
}

# Fundamental columns carried into the combined file (edit freely; see build_fu
ndamentals for all names)
FUND_COLS = [
    "eps_diluted", "eps_diluted_ttm", "eps_ttm_yoy", "revenue_ps_ttm", "revenue
_ttm_yoy",
```

```
    "cfo_ps_ttm", "fcf_ps_ttm", "dps_ttm",
    "book_value_ps", "cash_ps", "lt_debt_ps", "assets_ps", "liabilities_ps", "s
hares_out",
]
```

Helper functions: downloading
Nothing is downloaded yet; these cells only define functions.

Code cell 3 EDITED

```
def get_constituents() -> pd.DataFrame:
    """Scrape the current S&P 500 components table from Wikipedia."""
    for url in WIKI_URLS:
        page = requests.get(url, headers={"User-Agent": SEC_USER_AGENT}, timeou
t=30)
        if page.status_code != 200:
            continue
        try:
            tables = pd.read_html(StringIO(page.text))
        except ValueError:                # page has no tables at all
            continue
        for t in tables:
            cols = [" ".join(map(str, c)) if isinstance(c, tuple) else str(c) f
or c in t.columns]
            t.columns = cols
            find = lambda *words: next((c for c in cols if any(w in c.lower() f
or w in words)), None)
            tick, sub, name = find("symbol", "ticker"), find("sub"), find("secu
rity", "company")
            sec = next((c for c in cols if c != sub and ("sector" in c.lower()
or "industry" in c.lower())), None)
            added, cik = find("date added"), find("cik")
            if tick is None or sec is None or len(t) < MIN_ROWS:
                continue                  # e.g. the "changes" table further do
wn the page
            out = pd.DataFrame({
                "ticker": t[tick].astype(str).str.strip().str.upper(),
                "company": t[name] if name else "",
                "sector": t[sec],
                "sub_industry": t[sub] if sub else "",
                "date_added": pd.to_datetime(t[added], errors="coerce") if adde
d else pd.NaT,
                "cik_wiki": (pd.to_numeric(t[cik], errors="coerce").map(lambda
v: f"{int(v):010d}" if pd.notna(v) else None)
                             if cik else None),
            })
            return out.drop_duplicates("ticker").sort_values("ticker").reset_in
dex(drop=True)
    raise RuntimeError("Could not find the components table on Wikipedia; its l
ayout may have changed. Tried: "
```

```
                      + ", ".join(WIKI_URLS))


# Step 2. SEC helpers
def sec_get(url: str, session: requests.Session, retries: int = 4):
    """GET a JSON document from the SEC, staying well under its 10 requests/sec
ond limit."""
    for attempt in range(retries):
        r = session.get(url, timeout=60)
        if r.status_code == 200:
            time.sleep(0.15)
            return r.json()
        if r.status_code == 404:          # company has no XBRL facts
            return None
        time.sleep(2 ** attempt)          # 429 or 5xx: back off and retry
    r.raise_for_status()


def get_cik_map(session: requests.Session) -> dict:
    """Ticker -> 10-digit zero-padded CIK, from the SEC's own mapping file."""
    data = sec_get("https://www.sec.gov/files/company_tickers.json", session)
    return {v["ticker"].upper(): f'{int(v["cik_str"]):010d}' for v in data.valu
es()}


def get_companyfacts(cik: str, session: requests.Session):
    return sec_get(f"https://data.sec.gov/api/xbrl/companyfacts/CIK{cik}.json",
session)


# Step 3. Prices, splits, dividends
# >>> TEAM 2 EDIT (14 Oct 2026): smaller batches are gentler on Yahoo (fewer ra
te-limit failures)
# ORIGINAL: def download_prices(tickers: list, batch: int = 100) -> pd.DataFra
me:
def download_prices(tickers: list, batch: int = 25) -> pd.DataFrame:
# <<< END TEAM 2 EDIT
    """Daily bars in long format. `close` is split-adjusted but not dividend-ad
justed; `adj_close` is both."""
    start = (pd.Timestamp.today().normalize() - pd.DateOffset(years=LOOKBACK_YE
ARS)).strftime("%Y-%m-%d")
    frames, todo = {}, list(tickers)
    for attempt in range(2):              # the second pass retries anything th
at came back empty
        for i in range(0, len(todo), batch):
            chunk = todo[i:i + batch]
            yf_names = {t: t.replace(".", "-") for t in chunk}        # Yahoo w
rites BRK.B as BRK-B
```

```
            raw = yf.download(list(yf_names.values()), start=start, interval="1
d", auto_adjust=False,
                              actions=True, group_by="ticker", threads=True, pr
ogress=False)
            if raw is not None and not raw.empty:
                for t, yt in yf_names.items():
                    if yt not in raw.columns.get_level_values(0):
                        continue
                    df = raw[yt].dropna(subset=["Close"]).copy()
                    if df.empty:
                        continue
                    df.index = pd.to_datetime(df.index).tz_localize(None)
                    df.columns = [c.lower().replace(" ", "_") for c in df.colum
ns]
                    df.insert(0, "ticker", t)
                    frames[t] = df.rename_axis("date").reset_index()
            print(f"  prices: {len(frames)}/{len(tickers)} tickers")
            time.sleep(1)
        todo = [t for t in tickers if t not in frames]
        if not todo:
            break
    return pd.concat(frames.values(), ignore_index=True)
```

Helper functions: cleaning SEC data
These turn raw filing data into one clean row per company per quarter. They handle what a naive download gets wrong:

- **Year-to-date figures.** Cash flow is reported year-to-date and the fourth quarter is never reported on its own, so single quarters are derived by differencing.
- **Stock splits.** Old filings are rescaled to today's share count.
- **Filer unit errors.** Some filings tag share counts in thousands or millions as if they were single shares, or EPS multiplied up. `reconcile` checks that shares x EPS = net income and, where they disagree by a power of 1,000, uses the share price to decide which side is wrong.
- **Stray facts.** Only quarterly and annual reports are used, and every period must end on a balance-sheet date, so pro-forma periods and acquired companies' figures are left out.
- **Untagged EPS.** Where a filer does not tag EPS for a quarter, it is calculated as net income / diluted shares.

Code cell 4 Original, unchanged

```
def split_factor_after(dates: pd.Series, splits: pd.Series) -> pd.Series:
    """Cumulative split ratio of all splits that happened after each date (1.0
if none)."""
    f = pd.Series(1.0, index=dates.index)
    for d, ratio in splits.items():
        f[dates < d] *= ratio
    return f
```

```
def load_facts(cf: dict, key: str, unit: str, splits: pd.Series, per_share=Fals
e, share_count=False,
               taxonomy="us-gaap", split_ref="filed_last", largest=False) -> p
d.DataFrame:
    """
    One row per reporting period for a concept: start, end, val, filed_first, f
iled_last.

    - Merges the alternative tags in TAGS[key]; earlier tags win when two cover
the same period
      (or, with largest=True, the biggest value wins: used for revenue, where s
ome tags are sub-totals).
    - The same period appears in several filings (original + later comparative
s). We keep the
      most recently filed value, and remember the first filing date for point-i
n-time use.
    - Per-share values and share counts are put on today's share basis: a filin
g made before
      a split reports pre-split numbers, so we rescale by every split after tha
t filing date.
    """
    frames = []
    for prio, tag in enumerate(TAGS[key]):
        rows = cf.get("facts", {}).get(taxonomy, {}).get(tag, {}).get("units",
{}).get(unit)
        if rows:
            frames.append(pd.DataFrame(rows).assign(prio=prio))
    cols = ["start", "end", "val", "filed_first", "filed_last"]
    if not frames:
        return pd.DataFrame({c: pd.Series(dtype="float64" if c == "val" else "d
atetime64[ns]") for c in cols})
    d = pd.concat(frames, ignore_index=True)
    # quarterly and annual reports only: other filings (8-K exhibits, proxy sta
tements, merger documents)
    # can carry another company's figures or rounded repeats of old ones
    d = d[d["form"].str.startswith("10-", na=False) | d["form"].isin(["20-F",
"20-F/A", "40-F", "40-F/A"])]
    if d.empty:
        return pd.DataFrame({c: pd.Series(dtype="float64" if c == "val" else "d
atetime64[ns]") for c in cols})
    if "start" not in d:
        d["start"] = None
    for c in ("start", "end", "filed"):
        d[c] = pd.to_datetime(d[c])
    d["val"] = d["val"].astype(float)
    d = (d.sort_values("filed").groupby(["prio", "start", "end"], dropna=False)
          .agg(val=("val", "last"), filed_first=("filed", "min"), filed_last=
("filed", "last"))
          .reset_index())
```

```
    d["filed_first"] = d.groupby(["start", "end"], dropna=False)["filed_firs
t"].transform("min")
    d = (d.sort_values("val") if largest else d.sort_values("prio", ascending=F
alse))
    d = d.drop_duplicates(["start", "end"], keep="last")
    if per_share or share_count:
        f = split_factor_after(d[split_ref], splits)
        d["val"] = d["val"] / f if per_share else d["val"] * f
    cutoff = pd.Timestamp.today().normalize() - pd.DateOffset(years=LOOKBACK_YE
ARS + 1)
    return d.loc[d["end"] >= cutoff, cols]


EMPTY = pd.Series(dtype=float, index=pd.DatetimeIndex([]))


def to_quarterly(d: pd.DataFrame) -> pd.Series:
    """
    Discrete-quarter values indexed by period end, for income statement and cas
h flow items.

    10-Qs report three-month figures for the income statement but only year-to-
date figures
    for cash flow, and 10-Ks report only the full year. So:
      1. use a three-month fact when one exists;
      2. otherwise difference consecutive year-to-date facts (6M - 3M, 9M - 6M,
FY - 9M);
      3. otherwise Q4 = full year minus the three quarters inside it.
    """
    if d.empty:
        return EMPTY.copy()
    d = d.sort_values(["start", "end"]).copy()
    d["days"] = (d["end"] - d["start"]).dt.days
    same_start = d.groupby("start")
    d["gap"] = same_start["end"].diff().dt.days
    d["dval"] = same_start["val"].diff()
    is_q = d["days"].between(*Q_DAYS)
    direct = d[is_q].drop_duplicates("end", keep="last").set_index("end")["va
l"]
    derived = (d[~is_q & d["gap"].between(*Q_DAYS)]
               .drop_duplicates("end", keep="last").set_index("end")["dval"])
    q = direct.combine_first(derived).sort_index()
    for r in d[d["days"].between(*FY_DAYS)].itertuples():
        inside = q[(q.index > r.start) & (q.index < r.end)]
        if r.end not in q.index and len(inside) == 3:
            q.loc[r.end] = r.val - inside.sum()
    return q.sort_index()
```

```
def to_quarterly_avg(d: pd.DataFrame) -> pd.Series:
    """Weighted-average share counts: three-month average where reported, else
the full-year average (Q4)."""
    if d.empty:
        return EMPTY.copy()
    d = d.assign(days=(d["end"] - d["start"]).dt.days).sort_values("days")
    direct = d[d["days"].between(*Q_DAYS)].drop_duplicates("end", keep="last").
set_index("end")["val"]
    longest = d.drop_duplicates("end", keep="last").set_index("end")["val"]
    return direct.combine_first(longest).sort_index()


def cover_page_shares(cover: pd.DataFrame, quarter_ends: pd.DatetimeIndex) -> p
d.Series:
    """The cover-page share count is dated a few weeks after quarter end; assig
n it to that quarter."""
    out = pd.Series(np.nan, index=quarter_ends)
    for r in cover.sort_values("end").itertuples():
        prior = quarter_ends[quarter_ends <= r.end]
        if len(prior) and (r.end - prior[-1]).days <= 120:
            out[prior[-1]] = r.val
    return out


def ttm(s: pd.Series) -> pd.Series:
    """Sum of the last four reported quarters, only where those four quarters a
re actually consecutive."""
    v = s.dropna()
    span = pd.Series(v.index, index=v.index).diff(3).dt.days
    return v.rolling(4, min_periods=4).sum().where(span.between(240, 310)).rein
dex(s.index)


def yoy(s: pd.Series) -> pd.Series:
    """Growth versus the same quarter one year earlier; blank when the base is
zero or negative."""
    v = s.dropna()
    span = pd.Series(v.index, index=v.index).diff(4).dt.days
    base = v.shift(4).where(span.between(350, 380))
    return (v / base - 1).where(base > 0).reindex(s.index)


def snap_to_quarter_ends(d: pd.DataFrame, quarter_ends: pd.DatetimeIndex) -> p
d.DataFrame:
    """
    A fiscal period ends on a balance-sheet date. A fact dated a few days off o
ne belongs to that
    period, so its end date is moved onto it. A fact ending between two quarter
ends is a stray
```

```
    (a pro-forma period, a part-quarter since an acquisition) and is dropped. F
acts far from any
    balance sheet come from before the company filed its own reports, and are k
ept.
    """
    if d.empty or len(quarter_ends) == 0:
        return d
    ends = pd.DatetimeIndex(d["end"])
    nearest = quarter_ends[quarter_ends.get_indexer(ends, method="nearest")]
    off = np.abs((ends - nearest).days)
    d = d.assign(end=ends.where(off > 7, nearest), _off=off)[(off <= 7) | (off
> 60)]
    return d.sort_values("_off").drop_duplicates(["start", "end"]).drop(columns
="_off")


def rescale(x: pd.Series, ref: pd.Series, keep_unexplained: bool = False) -> p
d.Series:
    """
    Put a share count back on the scale of `ref`. Filers sometimes tag shares i
n thousands or
    millions as if they were single shares, so the error is a power of 1,000. V
alues within 10x of
    ref are left alone, and values that a power of 1,000 brings within 3x of re
f are rescaled.
    Anything else is blanked (or left as it is with keep_unexplained=True).
    """
    ratio = x / ref
    snapped = x / 10.0 ** (3 * np.round(np.log10(ratio.where(ratio > 0)) / 3))
    fits = (snapped / ref).between(1 / 3, 3)
    fine = ratio.between(0.1, 10) | ref.isna() | (keep_unexplained & ~fits)
    return x.where(fine, snapped.where(fits))


def reconcile(q: pd.DataFrame, price) -> pd.DataFrame:
    """
    Repair unit errors in the filings. Net income, EPS and the share count have
to agree:
    shares x EPS = net income. When they are out by a power of 1,000 (or EPS is
absurdly large),
    the filer tagged one of them in the wrong units. The share price says whic
h: the right version
    gives a believable market value ($100m to $10tn) and an EPS no bigger than
3x the share price.
    Adds a `fix` column saying what was corrected ("shares", "eps" or "").
    """
    S, E, N = q["shares_dil"].copy(), q["eps_diluted"], q["net_income"]
    implied = (N / E).where((E.abs() >= 0.01) & (N / E > 0))       # share coun
t implied by net income / EPS
```

```
    ratio = S / implied
    off = ratio.notna() & ~ratio.between(0.1, 10)
    usual = S[ratio.between(0.1, 10)].median()                      # this compa
ny's normal share count
    q["fix"] = ""
    # quarters with no EPS cannot be cross-checked, so compare them with the co
mpany's normal count
    unchecked = implied.isna() & S.notna()
    if unchecked.any() and pd.notna(usual):
        fixed = rescale(S[unchecked], pd.Series(usual, index=S[unchecked].inde
x), keep_unexplained=True)
        q.loc[unchecked, "shares_dil"] = fixed
        q.loc[fixed.index[fixed != S[unchecked]], "fix"] = "shares"
    if price is None or len(price) == 0:
        return q
    S = q["shares_dil"].copy()
    px = price.reindex(price.index.union(q.index)).sort_index().ffill().bfill
().reindex(q.index)
    believable = lambda shares, eps: (px * shares).between(1e8, 1e13) & (eps.ab
s() <= 3 * px)
    S_fixed = rescale(S, implied)                                   # blank unle
ss a power of 1,000 explains the gap
    E_fixed = N / S
    suspect = (off & S_fixed.notna()) | (E.abs() > 3 * px)
    if not suspect.any():
        return q
    a_ok = believable(S_fixed, E)                                   # version A:
the share count was wrong
    b_ok = believable(S, E_fixed)                                   # version B:
EPS was wrong
    in_line = (S / usual).between(1 / 3, 3)                         # share coun
t looks like every other quarter
    fix_eps = suspect & b_ok & (~a_ok | in_line)
    fix_shares = suspect & a_ok & ~fix_eps
    q.loc[fix_shares, "shares_dil"] = S_fixed[fix_shares]
    q.loc[fix_eps, "eps_diluted"] = E_fixed[fix_eps]
    q.loc[fix_shares, "fix"] = "shares"
    q.loc[fix_eps, "fix"] = "eps"
    return q


def build_fundamentals(cf: dict, splits: pd.Series, price: pd.Series = None) ->
pd.DataFrame:
    """
    Quarterly per-share table for one company. Index = fiscal quarter end.
    `price` is the daily close (split-adjusted), used only to sanity-check shar
e counts and EPS.
    """
    F = lambda k, unit, **kw: load_facts(cf, k, unit, splits, **kw)
```

```
    # balance sheet first: its dates define the fiscal quarter ends that everyt
hing else must line up with
    bs = {k: F(k, "USD").set_index("end")["val"].astype(float) for k in ("equit
y", "assets", "liabilities", "cash", "lt_debt")}
    quarter_ends = bs["assets"].index.union(bs["equity"].index).unique().sort_v
alues()
    P = lambda k, unit, **kw: snap_to_quarter_ends(F(k, unit, **kw), quarter_en
ds)     # facts that cover a period

    eps_raw = P("eps_diluted", "USD/shares", per_share=True)
    ni_raw = P("net_income", "USD")
    q = pd.DataFrame({
        "eps_diluted": to_quarterly(eps_raw),
        "eps_basic":   to_quarterly(P("eps_basic", "USD/shares", per_share=Tru
e)),
        "revenue":     to_quarterly(P("revenue", "USD", largest=True)),
        "net_income":  to_quarterly(ni_raw),
        "cfo":         to_quarterly(P("cfo", "USD")),
        "capex":       to_quarterly(P("capex", "USD")),
        "dps":         to_quarterly(P("dps", "USD/shares", per_share=True)),
        "shares_dil":  to_quarterly_avg(P("shares_dil", "shares", share_count=T
rue)),
        "shares_bas":  to_quarterly_avg(P("shares_bas", "shares", share_count=T
rue)),
    })
    q = q.dropna(subset=["eps_diluted", "net_income", "cfo"], how="all")
    if q.empty:
        return q
    q.index = pd.DatetimeIndex(q.index)

    # balance sheet items are point-in-time: just look them up at each quarter
end
    for k, v in bs.items():
        q[k] = v.reindex(q.index)
    # Shares outstanding, best available: balance sheet -> filing cover page ->
weighted average.
    # If the reported count is far from the weighted average (e.g. only one sha
re class was
    # tagged), trust the weighted average, which always covers every class.
    eop = F("shares_eop", "shares", share_count=True).set_index("end")["val"].r
eindex(q.index)
    cover = cover_page_shares(F("shares_cover", "shares", share_count=True, tax
onomy="dei", split_ref="end"), q.index)
    q["shares_dil"] = q["shares_dil"].fillna(q["shares_bas"])
    q = reconcile(q, price)                                          # fix share co
unts / EPS tagged in the wrong units
    q["shares_bas"] = rescale(q["shares_bas"], q["shares_dil"])
    q.loc[q["fix"] == "eps", "eps_basic"] = q["net_income"] / q["shares_bas"]
    avg = q["shares_bas"].fillna(q["shares_dil"])
```

```
    reported = rescale(eop, avg).fillna(rescale(cover, avg))
    q["shares_out"] = reported.where((reported / avg - 1).abs().lt(0.15) | avg.
isna(), avg).fillna(avg)
    q["shares_dil"] = q["shares_dil"].fillna(q["shares_out"])   # some filers o
mit the quarterly average

    # date the market first saw each quarter (use this, not period_end, when jo
ining to prices)
    first_seen = pd.concat([eps_raw, ni_raw]).groupby("end")["filed_first"].min
()
    q.insert(0, "filed", first_seen.reindex(q.index))

    # Some filers tag EPS only per share class, or leave it out of the annual r
eport, so Q4 cannot be
    # derived. EPS is net income / diluted shares by definition, so fill those
gaps that way.
    q["eps_calc"] = q["eps_diluted"].isna() & (q["net_income"] / q["shares_di
l"]).notna()
    q["eps_diluted"] = q["eps_diluted"].fillna(q["net_income"] / q["shares_di
l"])
    q["eps_basic"] = q["eps_basic"].fillna(q["net_income"] / q["shares_bas"])

    # trailing twelve months
    q["eps_diluted_ttm"] = ttm(q["eps_diluted"])
    q["eps_ttm_yoy"] = yoy(q["eps_diluted_ttm"])
    q["revenue_ttm"] = ttm(q["revenue"])
    q["revenue_ttm_yoy"] = yoy(q["revenue_ttm"])
    q["cfo_ttm"] = ttm(q["cfo"])
    q["fcf_ttm"] = q["cfo_ttm"] - ttm(q["capex"].fillna(0))
    q["dps_ttm"] = ttm(q["dps"].fillna(0))

    # per share
    q["revenue_ps_ttm"] = q["revenue_ttm"] / q["shares_dil"]
    q["cfo_ps_ttm"] = q["cfo_ttm"] / q["shares_dil"]
    q["fcf_ps_ttm"] = q["fcf_ttm"] / q["shares_dil"]
    q["book_value_ps"] = q["equity"] / q["shares_out"]
    q["cash_ps"] = q["cash"] / q["shares_out"]
    q["lt_debt_ps"] = q["lt_debt"] / q["shares_out"]
    q["assets_ps"] = q["assets"] / q["shares_out"]
    q["liabilities_ps"] = q["liabilities"] / q["shares_out"]
    return q.rename_axis("period_end")
```

1. Constituents and sector

Code cell 5 Original, unchanged

```
assert "example.com" not in SEC_USER_AGENT, "Set SEC_USER_AGENT in the settings
cell to your own name and email."
session = requests.Session()
```

```
session.headers.update({"User-Agent": SEC_USER_AGENT, "Accept-Encoding": "gzip, deflate"})

cons = get_constituents()
sec_cik = cons["ticker"].str.replace(".", "-", regex=False).map(get_cik_map(session))
cons["cik"] = sec_cik.fillna(cons["cik_wiki"])          # SEC's own mapping first, Wikipedia's as back-up
cons = cons.drop(columns="cik_wiki")
print(len(cons), "tickers,", cons["sector"].nunique(), "sectors,", cons["cik"].isna().sum(), "without an SEC ID")
cons.head()
```

2. Daily prices
Downloaded in batches of 100 tickers. Eight years are pulled so that stock splits and look-backs are covered; the file keeps the last five.

Code cell 6 EDITED

```
px_all = download_prices(cons["ticker"].tolist())

# >>> TEAM 2 EDIT (14 Oct 2026): re-download tickers that are missing or behind the latest date
# (Yahoo can silently drop the newest day when it rate-limits; seen on 14 Oct 2026 for 7 tickers)
for attempt in range(3):
    last = px_all.groupby("ticker")["date"].max()
    behind = sorted(last[last < last.max()].index) + sorted(set(cons["ticker"]) - set(px_all["ticker"]))
    if not behind:
        break
    print(f"Retry {attempt + 1}: {len(behind)} tickers behind -> {behind[:10]}")
    time.sleep(60)                                      # give Yahoo a minute
    px_all = px_all[~px_all["ticker"].isin(behind)]
    px_all = pd.concat([px_all, download_prices(behind, batch=10)], ignore_index=True)
# <<< END TEAM 2 EDIT
no_px = sorted(set(cons["ticker"]) - set(px_all["ticker"]))
print(f"{len(px_all):,} rows, {px_all['ticker'].nunique()} tickers, {px_all['date'].min():%Y-%m-%d} to {px_all['date'].max():%Y-%m-%d}")
print("No price data:", ", ".join(no_px) or "none")
```

3. Quarterly fundamentals from the SEC
One request per company, typically 5 to 10 minutes. If it is interrupted or a few companies fail, just run this cell again:
it keeps what it already has and only fetches what is missing.

Code cell 7 Original, unchanged

```
try:
```

```
    fund_frames                                     # survives re-runs of this ce
ll
except NameError:
    fund_frames = {}

failed = {}
split_rows = px_all[px_all["stock_splits"] > 0]
close_by_ticker = {t: g.set_index("date")["close"] for t, g in px_all.groupby
("ticker")}   # to sanity-check share counts
groups = cons.dropna(subset=["cik"]).groupby("cik")["ticker"].apply(list)
# share classes of one company share one ID
for i, (cik, tickers) in enumerate(groups.items(), 1):
    if i % 50 == 0:
        print(f"  {i}/{len(groups)} companies")
    if all(t in fund_frames for t in tickers):
        continue
    try:
        facts = get_companyfacts(cik, session)
        for t in tickers:
            if not facts:
                fund_frames[t] = None
                continue
            splits = split_rows.loc[split_rows["ticker"] == t].set_index("dat
e")["stock_splits"]
            if t in SHARE_CLASS_RATIO:            # treat the share-class ratio
like a split that applies to every filing
                splits = pd.concat([splits, pd.Series({pd.Timestamp("2200-01-0
1"): float(SHARE_CLASS_RATIO[t])})])
            f = build_fundamentals(facts, splits, close_by_ticker.get(t))
            fund_frames[t] = f.reset_index().assign(ticker=t) if not f.empty el
se None
    except Exception as e:                          # one bad company should not
stop the other 499
        failed[tickers[0]] = repr(e)[:120]

fund = pd.concat([f for f in fund_frames.values() if f is not None], ignore_ind
ex=True)
no_data = sorted(set(cons["ticker"]) - set(fund["ticker"]))
print(f"{len(fund):,} ticker-quarters for {fund['ticker'].nunique()} tickers")
print("No SEC quarterly data:", ", ".join(no_data) or "none")
if failed:
    print("Failed (run this cell again to retry):", failed)

repaired = fund[fund["fix"] != ""].groupby("fix")["ticker"].unique()
for what, names in repaired.items():
    print(f"Unit errors corrected in {what}: {len(names)} tickers ({', '.join(s
orted(names))})")
print("EPS calculated as net income / diluted shares where the filer did not ta
g it:",
```

```
      fund.loc[fund["eps_calc"] == True, "ticker"].nunique(), "tickers")
```

4. Combine into one table
Each daily price row gets the most recent quarter **that had already been filed on that day** (`merge_asof` on the filing date).
Quarterly values therefore repeat on every row until the next filing. That is intended, and it keeps the file free of look-ahead bias.

Code cell 8 Original, unchanged

```
# daily measures that need history before the five-year window
px = px_all.sort_values(["ticker", "date"]).copy()
g = px.groupby("ticker")
px["chg_pct"] = g["close"].pct_change() * 100
px["rel_vol"] = px["volume"] / g["volume"].transform(lambda v: v.shift(1).rolli
ng(10).mean())
px["div_ttm"] = g["dividends"].transform(lambda d: d.rolling(252, min_periods=
1).sum())
px = px[px["date"] >= px["date"].max() - pd.DateOffset(years=PRICE_YEARS)].copy
()

# fundamentals keyed by the date they became public
f = (fund.dropna(subset=["filed"])[["ticker", "filed", "period_end"] + FUND_COL
S]
         .rename(columns={"filed": "fund_filed", "period_end": "fiscal_period_e
nd"}))
px["date"] = px["date"].astype("datetime64[ns]")
f["fund_filed"] = f["fund_filed"].astype("datetime64[ns]")

panel = pd.merge_asof(px.sort_values("date"), f.sort_values(["fund_filed", "fis
cal_period_end"]),
                      left_on="date", right_on="fund_filed", by="ticker")
panel = panel.merge(cons[["ticker", "company", "sector", "sub_industry", "date_
added"]], on="ticker", how="left")

# daily valuation: today's price against the latest filed fundamentals
positive = lambda s: s.where(s > 0)
panel["mkt_cap"] = panel["close"] * panel["shares_out"]
panel["pe"] = panel["close"] / positive(panel["eps_diluted_ttm"])
panel["p_cfo"] = panel["close"] / positive(panel["cfo_ps_ttm"])
panel["p_fcf"] = panel["close"] / positive(panel["fcf_ps_ttm"])
panel["p_book"] = panel["close"] / positive(panel["book_value_ps"])
panel["div_yield_pct"] = panel["div_ttm"] / panel["close"] * 100
panel["eps_ttm_yoy_pct"] = panel["eps_ttm_yoy"] * 100
panel["revenue_ttm_yoy_pct"] = panel["revenue_ttm_yoy"] * 100
print(panel.shape)
```

5. Format and export
Fixed column order, sensible rounding, ISO dates, sorted by ticker then date. This cell writes the CSV.

Code cell 9 EDITED

```
COLUMNS = [
    # identity
    "date", "ticker", "company", "sector", "sub_industry", "date_added",
    # daily market data
    "open", "high", "low", "close", "adj_close", "volume", "chg_pct", "rel_vo
l", "dividends", "stock_splits",
    # daily valuation
    "mkt_cap", "pe", "p_cfo", "p_fcf", "p_book", "div_yield_pct",
    # latest filed fundamentals, per share
    "fiscal_period_end", "fund_filed",
    "eps_diluted", "eps_diluted_ttm", "eps_ttm_yoy_pct", "revenue_ps_ttm", "rev
enue_ttm_yoy_pct",
    "cfo_ps_ttm", "fcf_ps_ttm", "dps_ttm",
    "book_value_ps", "cash_ps", "lt_debt_ps", "assets_ps", "liabilities_ps", "s
hares_out",
]
ROUND = {**{c: 4 for c in ["open", "high", "low", "close", "adj_close", "divide
nds", "eps_diluted", "eps_diluted_ttm",
                           "revenue_ps_ttm", "cfo_ps_ttm", "fcf_ps_ttm", "dps_t
tm", "book_value_ps", "cash_ps",
                           "lt_debt_ps", "assets_ps", "liabilities_ps"]},
         **{c: 2 for c in ["chg_pct", "rel_vol", "pe", "p_cfo", "p_fcf", "p_boo
k", "div_yield_pct",
                           "eps_ttm_yoy_pct", "revenue_ttm_yoy_pct"]},
         "mkt_cap": 0, "shares_out": 0}

out = panel[[c for c in COLUMNS if c in panel.columns]].round(ROUND)
for c in ("volume", "mkt_cap", "shares_out"):
    out[c] = out[c].astype("Int64")               # whole numbers, blanks allow
ed
out = out.sort_values(["ticker", "date"]).reset_index(drop=True)

out.to_csv(OUT_FILE, index=False, date_format="%Y-%m-%d")
if COMPRESS:
    out.to_csv(OUT_FILE + ".gz", index=False, date_format="%Y-%m-%d", compressi
on="gzip")
```

**# >>> TEAM 2 EDIT (14 Oct 2026): also write one Parquet file per year, in a dat
ed folder**

```
# Parquet keeps column types (no schema file needed) and loads into BigQuery p
artitioned by date.
# SP500_OUT can be set by Dagster; default is ./data
from pathlib import Path
run_dir = Path(os.environ.get("SP500_OUT", "data")) / pd.Timestamp.today().str
ftime("%Y-%m-%d")
run_dir.mkdir(parents=True, exist_ok=True)
for year, part in out.groupby(out["date"].dt.year):
    part.to_parquet(run_dir / f"sp500_panel_{year}.parquet", index=False)
```

```
print(f"Parquet written to {run_dir.resolve()}")
# <<< END TEAM 2 EDIT


import os
print(f"Saved {os.path.abspath(OUT_FILE)}")
print(f"{len(out):,} rows x {out.shape[1]} columns, {out['ticker'].nunique()} t
ickers, "
      f"{out['date'].min():%Y-%m-%d} to {out['date'].max():%Y-%m-%d}, {os.path.
getsize(OUT_FILE)/1e6:.0f} MB")
out.tail()
```

6. The screen
The screener view is simply the latest row for each ticker, sorted by market cap.

Code cell 10 EDITED

```
screen = (out[out["date"] == out["date"].max()]
          .sort_values("mkt_cap", ascending=False)
          [["ticker", "company", "sector", "close", "chg_pct", "volume", "rel_v
ol", "mkt_cap", "pe",
            "eps_diluted_ttm", "eps_ttm_yoy_pct", "div_yield_pct", "cfo_ps_tt
m", "book_value_ps"]]
          .reset_index(drop=True))
# >>> TEAM 2 EDIT (14 Oct 2026): show the plain table; the styled version needs
jinja2 and is display-only
screen.head(25)
# ORIGINAL (formatting only):
# screen.head(25).style.format({
#     "close": "{:,.2f}", "chg_pct": "{:+.2f}%", "volume": "{:,.0f}", "rel_vo
l": "{:.2f}",
#     "mkt_cap": lambda v: "" if pd.isna(v) else (f"{v/1e12:.2f} T" if v >= 1e
12 else f"{v/1e9:.2f} B"),
#     "pe": "{:.2f}", "eps_diluted_ttm": "{:.2f}", "eps_ttm_yoy_pct": "{:+.2
f}%", "div_yield_pct": "{:.2f}%",
#     "cfo_ps_ttm": "{:.2f}", "book_value_ps": "{:.2f}",
# }, na_rep="")
# <<< END TEAM 2 EDIT
```

7. Sector snapshot
One row per sector on the latest day in the file: how many stocks, the sector's total market cap, and its largest stock.
The count includes stocks with no market cap; the totals and the ranking cannot.

Code cell 11 Original, unchanged

```
latest = out[out["date"] == out["date"].max()]
ranked = latest.dropna(subset=["mkt_cap"]).sort_values("mkt_cap", ascending=Fal
se).groupby("sector")
sector_snapshot = pd.DataFrame({
    "number_of_stocks": latest.groupby("sector").size(),
```

```
    "sector_mkt_cap_billions": ranked["mkt_cap"].sum().astype(float) / 1e9,
    "largest_stock_ticker": ranked["ticker"].first(),
    "largest_company_name": ranked["company"].first(),
    "largest_stock_mkt_cap_billions": ranked["mkt_cap"].first().astype(float) /
1e9,
}).sort_values("number_of_stocks", ascending=False).round(2).reset_index()
print(f"As of {latest['date'].iloc[0]:%Y-%m-%d}")
sector_snapshot
```

8. Sanity checks
Run this after every export. Filing data contains errors, and one bad share count is enough to put a mid-sized company at the top of a ranking.
A jump in share count is not always an error: a large merger shows up here too (Warner Bros. Discovery in 2022, for example).

Code cell 12 Original, unchanged

```
latest = out[out["date"] == out["date"].max()]
cap_bn = latest.set_index("ticker")["mkt_cap"].astype(float) / 1e9
print("No market cap:", ", ".join(cap_bn[cap_bn.isna()].index) or "none")
odd = cap_bn[(cap_bn < 1) | (cap_bn > 8000)]
print("Market cap below $1bn or above $8tn:", ", ".join(f"{t} ({v:,.1f}bn)" for
t, v in odd.items()) or "none")

quarters = out.drop_duplicates(["ticker", "fiscal_period_end"]).dropna(subset=
["shares_out"])
step = quarters["shares_out"].astype(float) / quarters.groupby("ticker")["share
s_out"].shift().astype(float)
jumps = quarters[(step > 3) | (step < 1 / 3)]
print("Share count changing more than 3x between quarters:",
      ", ".join(f"{r.ticker} ({r.fiscal_period_end:%Y-%m})" for r in jumps.iter
tuples()) or "none")

no_eps = latest.loc[latest["eps_diluted_ttm"].isna(), "ticker"]
print(f"No trailing EPS ({len(no_eps)} tickers):", ", ".join(no_eps) or "none")
```

Reading the file back
```python
panel = pd.read_csv("sp500_panel.csv", parse_dates=["date", "date_added",
"fiscal_period_end", "fund_filed"])
one_stock = panel[panel["ticker"] == "JPM"]                    # daily history for one
name
quarters = panel.drop_duplicates(["ticker", "fiscal_period_end"])  # one row per
ticker-quarter
in_index = panel[panel["date"] >= panel["date_added"]]          # only days after
the stock joined the index
```

Notes
- The list is **today's** S&P 500 members, so the history excludes companies that left the index (survivorship bias).
- For banks, insurers and REITs, free cash flow and long-term debt per share are not meaningful; EPS and book value are.

- Companies that report per share class (Berkshire Hathaway, Visa and a few others) have no single share count in the SEC data, so their market cap and per-share columns are blank. A few partnership or multi-class structures (Blackstone, Interactive Brokers) come out understated. Step 8 lists the blanks.

Code cell 13 EDITED

```
# >>> TEAM 2 EDIT (14 Oct 2026): stop with an error if the download is incomplete,
# so a scheduled (Dagster) run is marked FAILED instead of "success with gaps".
import sys
last = out.groupby("ticker")["date"].max()
behind = last[last < last.max()]
print(f"{out['ticker'].nunique()} tickers, {len(behind)} behind the latest date: {list(behind.index)}")
if out["ticker"].nunique() < 495 or len(behind) > 5:
    print("FAILED: incomplete download")
    sys.exit(1)
print("Completeness check passed")
# <<< END TEAM 2 EDIT
```